

lab3-1 基于 UDP 服务设计可靠传输协议并编程实现

才子桓 2212700 密码科学与技术

实验要求

实验3-1： 利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：建立连接、差错检测、确认重传等。流量控制采用停等机制，完成给定测试文件的传输。

实验3-2： 在实验 3-1 的基础上，将停等机制改成基于滑动窗口的流量控制机制，采用固定窗口大小，支持累积确认，完成给定测试文件的传输。

实验3-3： 在实验 3-2 的基础上，选择实现一种拥塞控制算法，也可以是改进的算法，完成给定测试文件的传输。

实验3-4： 基于给定的实验测试环境，通过改变网路的延迟时间和丢包率，完成下面3组性能对比实验：

- (1) 停等机制与滑动窗口机制性能对比；
- (2) 滑动窗口机制中不同窗口大小对性能的影响；
- (3) 有拥塞控制和无拥塞控制的性能比较。

要求如下：

1. 实现单向传输。
2. 对于每一个任务要求给出详细的协议设计。
3. 给出实现的拥塞控制算法的原理说明。
4. 完成给定测试文件的传输，显示传输时间和平均吞吐率。
5. 性能测试指标包括吞吐率、文件传输时延等，给出图形结果并进行分析。
6. 完成详细的实验报告（每个任务完成一份）。
7. 编写的程序应该结构清晰，具有较好的可读性。
8. 提交程序源码、可执行文件和实验报告。

实验说明

本次实验为基于 UDP 服务设计可靠传输协议并编程实现，UDP 是无连接的协议，也不保证可靠交付，只在IP数据报服务至上增加了很少的功能，主要是复用和分用以及差错检测的功能。这适用于要求源主机以恒定速率发送数据，允许网络拥塞时丢失数据，却不允许数据有太大时延的实时应用。

在 UDP 协议上实现可靠传输协议，所以我选择模仿 TCP 协议进行实现。在建立连接与断开连接的过程模仿**三次握手**和**四次挥手**协议，在流量控制中采用**停等机制**，即一方未收到回应前不会发送下一条报文。差错检测则通过在报文格式中设置**校验和**来验证正确性，同时丢包情况下采用**超时重传**。

在此基础上，进行文件传输测试以及丢包情况下的超时重传。同时在 Client/Server 的日志输出中显示收到/发送数据包的序号、ACK、校验和，传输结束后显示传输时间与吞吐率。

实验环境

IDE: Visual Studio 2022

语言: c++ 14标准

lab3 测试文件

路由器 Router

本实验在 Windows 系统下进行。

实验分析及代码实现

本实验意在基于UDP服务设计可靠传输协议，我这里选择模仿 TCP 协议完成可靠传输协议。TCP 协议为一种面向广域网的通信协议，基于流的方式，面向连接的可靠通信方式。实验3-1中要求实现建立连接、差错检测、确认重传等功能，这里我仿照 TCP 数据包的格式创建 Message 结构体，其中建立连接我选择的是模仿 TCP 协议中创建连接的三次握手和四次挥手协议；差错检测选择通过首部校验和完成。根据 PPT 中的示意图，可以得知本次实验两个虚拟设备 Client 和 Server 即通信双方，实现单向传输即 Client 发出后由 Server 接收。

Client_ip = 192.168.10.1
Client_port = 12
Server_ip = 192.168.10.3
Server_port = 56

Client_ip = 192.168.10.1
Client_port = 12
Server_ip = 192.168.10.3
Server_port = 56



报文格式

在报文设计中我只添加了本次实验所需的报文头信息，包括源、目的 IP 地址和端口号，序列号 (Seq)，确认号 (Ack)，文件大小 (size)，标志位 (flag)，头部校验和 (checksum)，并均初始化为 0。

报文段我使用 data 来表示，Max_DATA_SIZE 是我在程序开头设置的包的最大字节数。由于实验提供的路由器 Router 最大能转发的是 15000 字节，在尝试后 15000 和 14999 均未实现转发，最后选择为 14000。同报文头一样，data 也用 0 填充。

```
#define Max_DATA_SIZE 14000

struct Message {
    int SrcIP, DestIP;
    short SrcPort, DestPort;
    int Seq;
    int Ack;
    int size;
    short flag;
    short checksum;
    BYTE data[Max_DATA_SIZE];

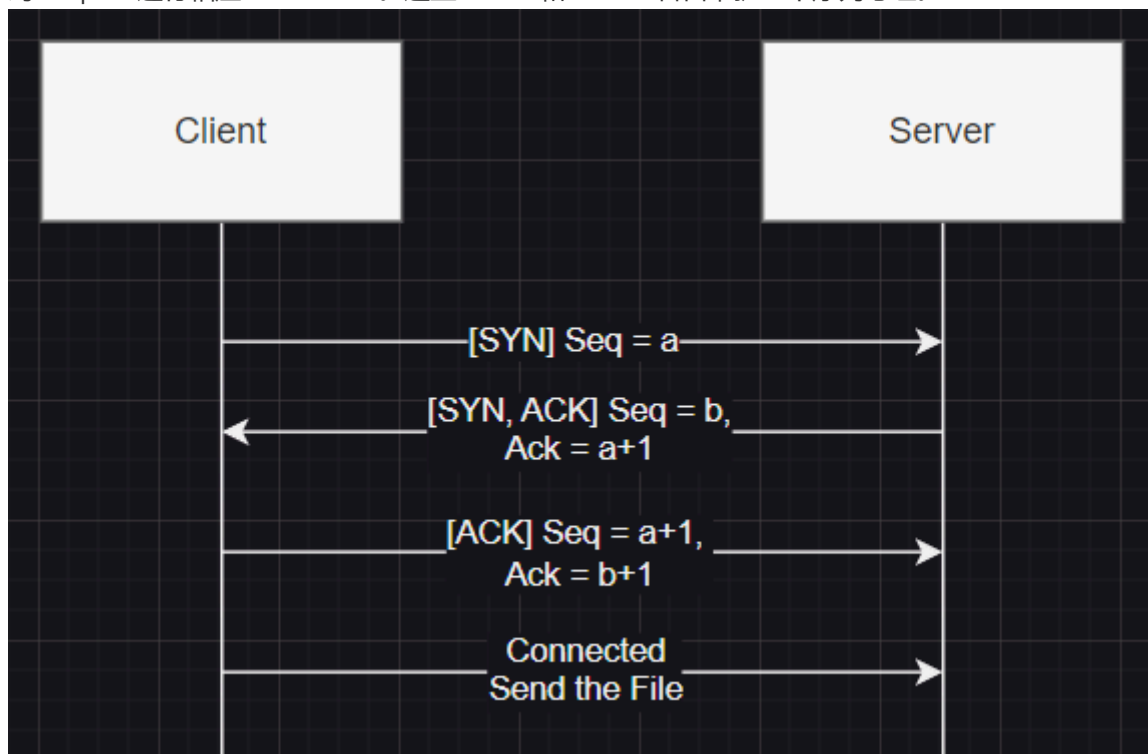
    Message() : SrcIP(0), DestIP(0), SrcPort(0), DestPort(0),
               Seq(0), Ack(0), size(0), flag(0), checksum(0) {
        memset(data, 0, sizeof(data));
    }

    ...
}
```

建立连接

建立连接：“三次握手”

连接建立我模仿 TCP 协议中的三次握手方式，Client 先向 Server 发送建立连接的请求，序列号为 a ，Server 收到这条请求报文后，发送响应 $Ack = a + 1$ ，同时 Server 也开启一个序列号 $Seq = b$ ，Client 再对 $Seq = b$ 进行相应 $Ack = b + 1$ 。这里 Client 和 Server 各自维护一个序列号组。



代码实现：

```
// Server
bool Three_Shakehands(SOCKET serverSocket, SOCKADDR_IN clientAddr) {
    int AddrLen = sizeof(clientAddr);
    Message msg1, msg2, msg3;
    int resendtimes = 0;

    while (1) {
        int recvByte = recvfrom(serverSocket, (char*)&msg1, sizeof(msg1), 0,
            (sockaddr*)&clientAddr, &AddrLen);

        if (recvByte > 0) {
            if (!(msg1.flag & SYN) || !msg1.check()) {
                Print("First Handshake failed.(Receive)");
                return false;
            }
        }
        Print("First Handshake succeeded.(Receive)");

        msg2.SrcPort = ServerPORT;
        msg2.DestPort = RouterPORT;
        msg2.Seq = init_seq;
        msg2.Ack = msg1.Seq + 1;
        msg2.flag += SYN;
        msg2.flag += ACK;
        msg2.setCheck();
```

```

        int sendByte = sendto(serverSocket, (char*)&msg2, sizeof(msg2), 0,
(sockaddr*)&clientAddr, AddrLen);
        clock_t msg2start = clock();

        if (sendByte == 0) {
            Print("Second Handshake failed.(Send)");
            return false;
        }
        Print("Second Handshake succeeded.(Send)");

        while (true) {
            int recvByte = recvfrom(serverSocket, (char*)&msg3, sizeof(msg3),
0, (sockaddr*)&clientAddr, &AddrLen);
            if (recvByte > 0) {
                if ((msg3.flag & ACK) && msg3.check() && (msg3.Ack ==
msg2.Seq + 1)) {
                    init_seq++;
                    Print("Third Handshake succeeded.(Receive)");
                    return true;
                }
                else {
                    Print("Third Handshake failed.(Receive)");
                    return false;
                }
            }

            if (clock() - msg2start > MAX_WAIT_TIME) {
                if (++resendtimes > MAX_SEND_TIMES) {
                    Print("Second Handshake resended times have reached max
num.");
                    return false;
                }
                cout << Get_Time() << " " << "Second Handshake Resend, time
" << resendtimes << endl;
                int sendByte = sendto(serverSocket, (char*)&msg2,
sizeof(msg2), 0, (sockaddr*)&clientAddr, AddrLen);
                msg2start = clock();
                if (sendByte > 0)
                    Print("Second Handshake resended successfully.");
                else
                    Print("Second Handshake resended failedly.");
            }
        }
    }
    return false;
}

// Client
bool Three_Shakehands(SOCKET clientSocket, SOCKADDR_IN serverAddr) {
    int AddrLen = sizeof(serverAddr);
    Message msg1, msg2, msg3;
    int resendtimes = 0;

    msg1.SrcPort = ClientPORT;
    msg1.DestPort = RouterPORT;

```

```

msg1.flag += SYN;
msg1.Seq = init_seq;
msg1.setCheck();

int sendByte = sendto(clientSocket, (char*)&msg1, sizeof(msg1), 0,
(sockaddr*)&serverAddr, AddrLen);
clock_t msg1start = clock();
if (sendByte > 0)
    Print("First Handshake succeeded.(Send)");

while (1) {
    int recvByte = recvfrom(clientSocket, (char*)&msg2, sizeof(msg2), 0,
(sockaddr*)&serverAddr, &AddrLen);
    if (recvByte > 0) {
        if ((msg2.flag & ACK) && (msg2.flag & SYN) && msg2.check() &&
(msg2.Ack == msg1.Seq + 1)) {
            Print("Second Handshake succeeded.(Receive)");
            break;
        }
        else
            Print("Second Handshake failed.(Receive)");
    }

    if (clock() - msg1start > MAX_WAIT_TIME) {
        if (++resendtimes > MAX_SEND_TIMES) {
            Print("First Handshake has reach max times.");
            return false;
        }
        cout << Get_Time() << "    " << "First Handshake Resend, time " <<
resendtimes << endl;
        sendByte = sendto(clientSocket, (char*)&msg1, sizeof(msg1), 0,
(sockaddr*)&serverAddr, AddrLen);
        msg1start = clock();
        if (sendByte <= 0) {
            Print("First Handshake resended failed.");
            return false;
        }
    }
}

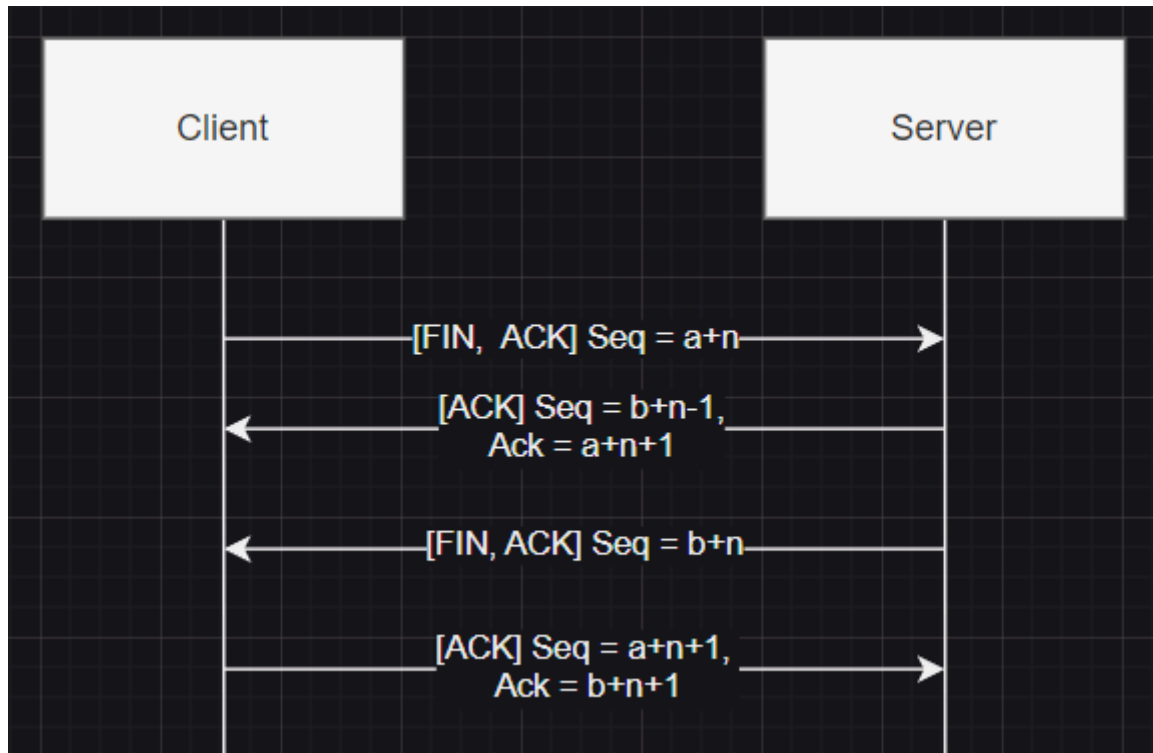
msg3.SrcPort = ClientPORT;
msg3.DestPort = RouterPORT;
msg3.flag += ACK;
msg3.Seq = ++init_seq;
msg3.Ack = msg2.Seq + 1;
msg3.setCheck();

sendByte = sendto(clientSocket, (char*)&msg3, sizeof(msg3), 0,
(sockaddr*)&serverAddr, AddrLen);
if (sendByte == 0) {
    Print("Third Handshake failed.(Send)");
    return false;
}
Print("Third Handshake success.(Send)");
return true;
}

```

断开连接：“四次挥手”

断开连接也是模仿 TCP 协议的四次挥手方式，Client 首先向 Server 发送断连请求（包含 FIN 标志）Seq = n，Server 收到 Client 发来的第一次挥手报文后，需要回复一个 ACK 报文，在第一次挥手的 Seq 序列号上加一。同理 Server 再发送包含 FIN 标识的第三次挥手，表示也可以断开连接。Client 接收到挥手请求后会回复 ACK 报文。



代码实现:

```
// Server
bool Four_Wavehands(SOCKET serverSocket, SOCKADDR_IN clientAddr) {
    int AddrLen = sizeof(clientAddr);
    Message msg1, msg2, msg3, msg4;
    int resendtimes = 0;

    while (1) {
        int recvByte = recvfrom(serverSocket, (char*)&msg1, sizeof(msg1), 0,
            (sockaddr*)&clientAddr, &AddrLen);
        if (recvByte == 0) {
            Print("First Handwave failed.(Receive)");
            return false;
        }
        else if (recvByte > 0) {
            if (!(msg1.flag & FIN) || !(msg1.flag & ACK) || !msg1.check()) {
                Print("First Handwave failed.(Receive)");
                return false;
            }
            Print("First Handwave succeeded.(Receive)");

            msg2.SrcPort = ServerPORT;
            msg2.DestPort = RouterPORT;
            msg2.Seq = init_seq++;
            msg2.Ack = msg1.Seq + 1;
            msg2.flag += ACK;
            msg2.setCheck();
```

```

        int sendByte = sendto(serverSocket, (char*)&msg2, sizeof(msg2), 0,
(sockaddr*)&clientAddr, AddrLen);
        clock_t msg2start = clock();
        if (sendByte == 0) {
            Print("Second Handwave failed.(Send)");
            return false;
        }
        Print("Second Handwave succeeded.(Send)");
        break;
    }
}

msg3.SrcPort = ServerPORT;
msg3.DestPort = RouterPORT;
msg3.flag += FIN;
msg3.flag += ACK;
msg3.Seq = init_seq++;
msg3.setCheck();
int sendByte = sendto(serverSocket, (char*)&msg3, sizeof(msg3), 0,
(sockaddr*)&clientAddr, AddrLen);
clock_t msg3start = clock();
if (sendByte == 0) {
    Print("Third Handwave failed.(Send)");
    return false;
}
Print("Third Handwave succeeded.(Send)");

while (1) {
    int recvByte = recvfrom(serverSocket, (char*)&msg4, sizeof(msg4), 0,
(sockaddr*)&clientAddr, &AddrLen);
    if (recvByte == 0) {
        Print("Forth Handwave failed.(Receive)");
        return false;
    }
    else if (recvByte > 0) {
        if ((msg4.flag & ACK) && msg4.check() && (msg4.Ack == msg3.Seq + 1))
        {
            Print("Forth Handwave succeeded.(Receive)");
            break;
        }
        else {
            Print("Forth Handwave failed.(Receive)");
            return false;
        }
    }
    if (clock() - msg3start > MAX_WAIT_TIME) {
        cout << Get_Time() << "    " << "Third Handwave Resend, time " <<
++resendtimes << endl;
        int sendByte = sendto(serverSocket, (char*)&msg3, sizeof(msg3), 0,
(sockaddr*)&clientAddr, AddrLen);
        msg3start = clock();
        if (sendByte > 0) {
            Print("Third Handshake resended successfully.");
            continue;
        }
        else {

```

```

        Print("Third Handshake resended failedly.");
    }
}
if (resendtimes == MAX_SEND_TIMES) {
    Print("Third Handshake resended times have reached max num.");
    return false;
}
}
Print("Server Connection has closed.");
return true;
}

// Client
bool Four_Wavehands(SOCKET clientSocket, SOCKADDR_IN serverAddr) {
    int AddrLen = sizeof(serverAddr);
    Message msg1, msg2, msg3, msg4;
    int resendtimes = 0;

    // Send first FIN message
    msg1.SrcPort = ClientPORT;
    msg1.DestPort = RouterPORT;
    msg1.flag += FIN;
    msg1.flag += ACK;
    msg1.Seq = ++init_seq;
    msg1.setCheck();
    int sendByte = sendto(clientSocket, (char*)&msg1, sizeof(msg1), 0,
(sockaddr*)&serverAddr, AddrLen);
    clock_t msg1start = clock();
    if (sendByte == 0) {
        Print("First Handwave failed.(Send)");
        return false;
    }
    Print("First Handwave succeeded.(Send)");

    while (1) {
        int recvByte = recvfrom(clientSocket, (char*)&msg2, sizeof(msg2), 0,
(sockaddr*)&serverAddr, &AddrLen);
        if (recvByte > 0) {
            if ((msg2.flag & ACK) && msg2.check() && (msg2.Ack == msg1.Seq + 1))
{
                Print("Second Handwave succeeded.(Receive)");
                break;
            }
        }

        if (clock() - msg1start > MAX_WAIT_TIME) {
            cout << Get_Time() << " " << "First Handwave Resend, time " <<
++resendtimes << endl;
            sendByte = sendto(clientSocket, (char*)&msg1, sizeof(msg1), 0,
(sockaddr*)&serverAddr, AddrLen);
            msg1start = clock();
            if (sendByte > 0) {
                Print("First Handwave resended successfully.");
                break;
            }
        }
    }
}

```

```

        else
            Print("First Handwave resended fialedly.");
    }
    if (resendtimes == MAX_SEND_TIMES) {
        Print("First Handwave resended times have reached max num.");
        return false;
    }
}

while (1) {
    int recvByte = recvfrom(clientSocket, (char*)&msg3, sizeof(msg3), 0,
(sockaddr*)&serverAddr, &AddrLen);
    if (recvByte > 0) {
        if ((msg3.flag & ACK) && (msg3.flag & FIN) && msg3.check()) {
            Print("Third Handwave succeeded.(Receive)");
            break;
        }
    }
}

// Send final ACK
msg4.SrcPort = ClientPORT;
msg4.DestPort = RouterPORT;
msg4.flag += ACK;
msg4.Seq = ++init_seq;
msg4.Ack = msg3.Seq + 1;
msg4.setCheck();
sendByte = sendto(clientSocket, (char*)&msg4, sizeof(msg4), 0,
(sockaddr*)&serverAddr, AddrLen);
if (sendByte == 0) {
    Print("Forth Handwave failed.(Send)");
    return false;
}
Print("Forth Handwave succeeded.(Send)");

// TIME_WAIT state
int tempclock = clock();
Print("Client in TIME_WAIT state");
Message tmp;
while (clock() - tempclock < 2 * MAX_WAIT_TIME) {
    int recvByte = recvfrom(clientSocket, (char*)&tmp, sizeof(tmp), 0,
(sockaddr*)&serverAddr, &AddrLen);
    if (recvByte > 0) {
        sendByte = sendto(clientSocket, (char*)&msg4, sizeof(msg4), 0,
(sockaddr*)&serverAddr, AddrLen);
        Print("Resent final ACK during TIME_WAIT");
    }
}
Print("Client successfully closed connection!");
return true;
}

```

差错检测

我们在报文格式设计中插入了两个函数 `check`、`setCheck` 来进行差错检测。

`check` 函数用于验证当前消息的校验和是否正确。如果传输过程中某些位被篡改，校验和会检测到不一致。

- 初始化校验和: `unsigned int sum = 0`; 初始化累加器 `sum` 为 0, 用于计算校验和。
- 数据转换为 16 位块: 使用 `unsigned short* msgStream = (unsigned short*)this`; 将整个 `Message` 结构体视为一个 `unsigned short` 的数组, 逐 16 位处理。
- 累加校验值:
 - 遍历结构体, 将每两个字节 (16 位) 累加到 `sum` 中。
 - 如果 `sum` 超过 16 位 (发生溢出), 则高 16 位与低 16 位相加, 这种处理称为“回卷”或“溢出折返”, 保证 `sum` 始终是 16 位。
- 校验结果: 最终, 如果 `sum` 的低 16 位与 `0xFFFF` 比较相等, 则校验成功, 返回 `true`。

`setCheck` 函数用于为当前消息计算校验和并设置到 `checknum` 字段, 保证消息可以通过 `check` 函数验证。

- 清零校验值: `this->checknum = 0`; 在计算前, 将 `checknum` 置为 0, 防止之前的校验值影响当前的计算。
- 数据转换为 16 位块: 和 `check` 函数类似, 将整个 `Message` 结构体视为 `unsigned short` 的数组逐 16 位处理。
- 累加校验值: 和 `check` 函数类似, 逐 16 位累加到 `sum` 中, 并处理溢出。
- 取反赋值: 将累加结果的低 16 位取反, `~(sum & 0xFFFF)`, 并赋值给 `checknum`。这一步是为了满足校验和的验证条件。

代码如下:

```
struct Message {
    ...

    bool check() {
        unsigned int sum = 0;
        unsigned short* msgStream = (unsigned short*)this;

        for (int i = 0; i < sizeof(*this) / 2; i++) {
            sum += *msgStream++;
            if (sum & 0xFFFF0000) {
                sum &= 0xFFFF;
                sum++;
            }
        }
        return (sum & 0xFFFF) == 0xFFFF;
    };
    void setCheck() {
        this->checknum = 0;
        int sum = 0;
        unsigned short* msgStream = (unsigned short*)this;

        for (int i = 0; i < sizeof(*this) / 2; i++) {
            sum += *msgStream++;
```

```

        if (sum & 0xFFFF0000) {
            sum &= 0xFFFF;
            sum++;
        }
    }
    this->checksum = ~(sum & 0xFFFF);
};
};

```

超时重传

由于本次实验的流量控制采用**停等机制**，即发送方在发送一个数据包后，必须停止并等待接收方的确认（ACK）信号，只有在收到ACK后，才能发送下一个数据包。

这种方式确保了每个数据包都被确认接收，从而避免了数据的丢失或乱序。

连接建立的超时重传

Client 发送第一次握手，同时开始计时，等待 Server 发来的第二次握手。

如果计时器超时（超出在宏定义中设置的时间），Client 则会重新发送第一次握手。

如果重传次数超过在宏定义中设置的最大次数，则默认为失败，退出连接。

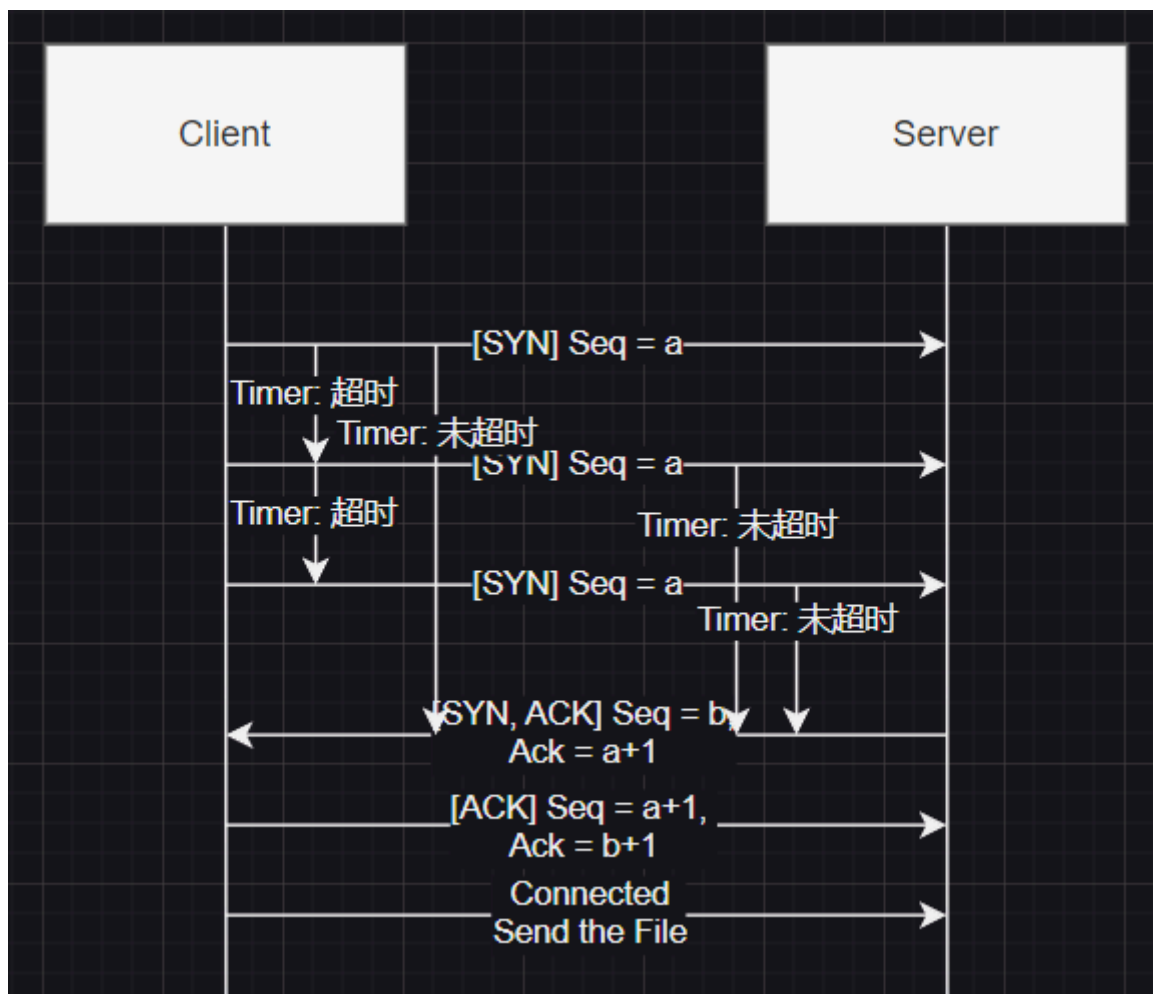
同 Client 原理一样，Server 接收到第一次握手后，发送第二条握手，并开始计时，等待第三条握手。

如果计时器超时（超出在宏定义中设置的时间），Server 则会重新发送第二次握手。若重传次数超过最大次数，则默认失败退出连接。

```

#define MAX_WAIT_TIME 5000
#define MAX_SEND_TIMES 50

```



连接断开的超时重传

类似于建立连接的超时重传。

Client 发送第一次挥手，同时开始计时，等待 Server 发来的第二次挥手。

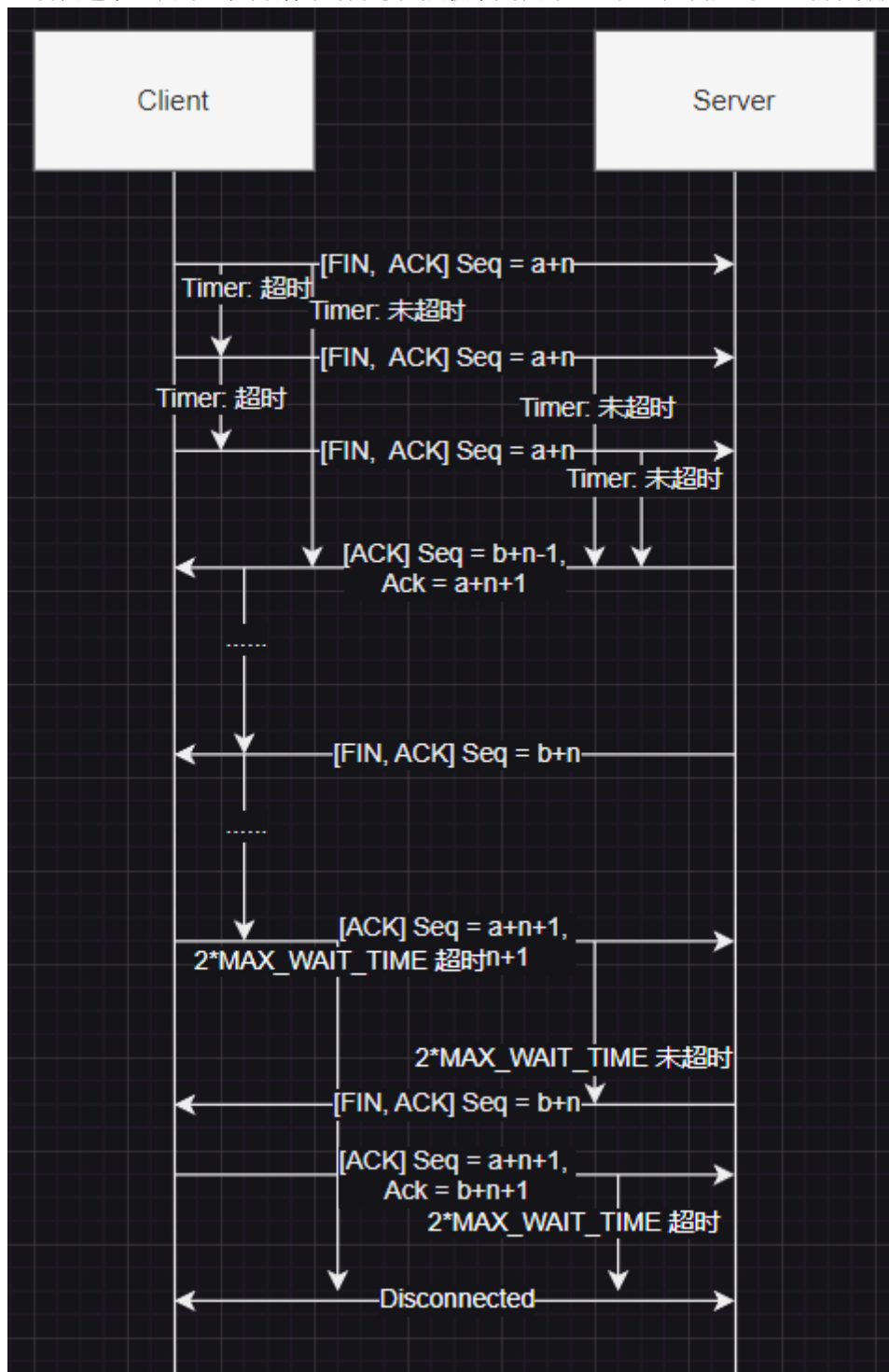
Client 等待第二次挥手，如果计时器超时（超出在宏定义中设置的时间），Client 则会重新发送第一次挥手。若重传次数超过最大次数，则默认失败退出连接。

Client 等待第三次挥手，接收到第三次挥手后，发送第四次挥手。

注意：由于 Client 并不知道 Server 是否正确收到了第四次挥手，所以 Client 发送完第四次挥手后，需要等待大于一个最长等待时间的时间，看是否接收到 Server 再次发送的第三次挥手的消息。若接收到。证明第三次挥手丢失，需要重传。

同理，Server 接收到第一次挥手后，发送第二次挥手。检查是否超时、重传次数是否超过最大次数。

之后发送第三次挥手，开始计时并等待接收第四次挥手。检查是否超时、重传次数是否超过最大次数。



文件传输的超时重传

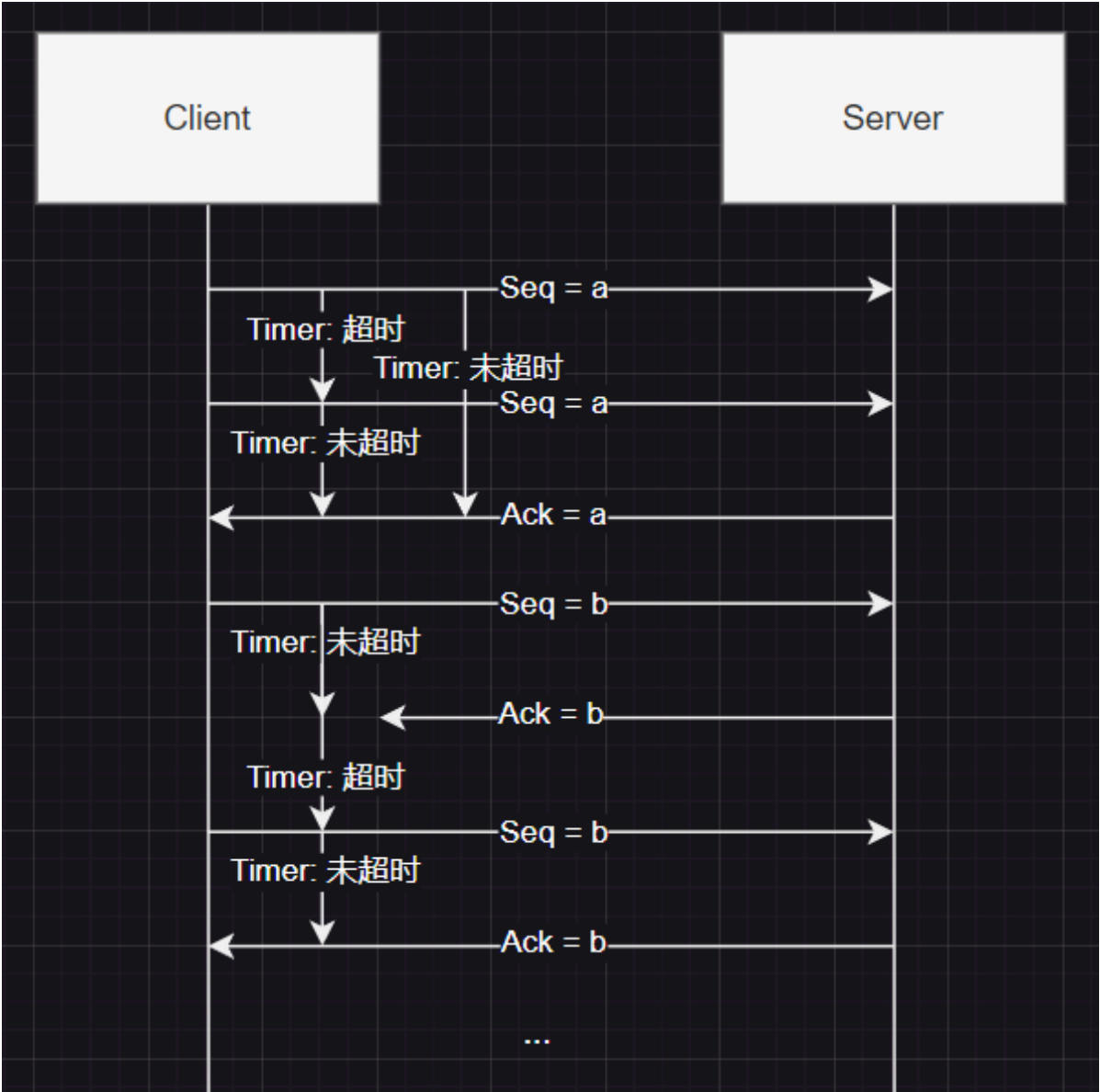
本次实验是实现从 Client 到 Server 的单向传输，采用停等协议。四个文件均大于单次传输最大字节数，所以文件的传输是需要将文件拆分成多个报文数据包进行传输。

报文数据包需要分别装载文件名字及大小、文件内容，其中装载文件内容的数据报文包还分为满载报文和未满载报文。

正常情况无丢包时，传输过程是没有失序问题的。此时 Client 和 Server 双方只需按照顺序接收和发送。Client 发送报文数据包并开始计时，等待接收 ACK 报文，若超时未接受到报文，则重传；若接收到，则继续发送下一个报文包。

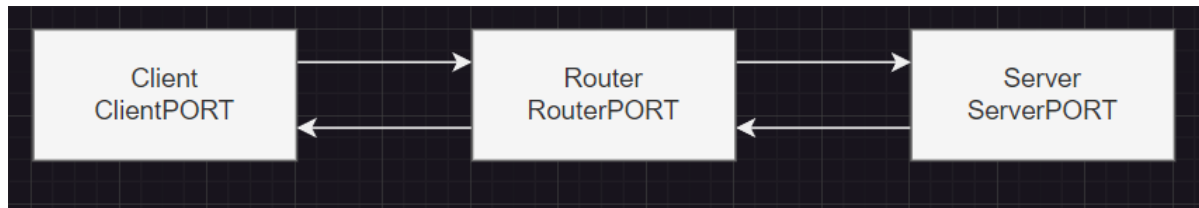
在测试过程中模拟数据包丢失情况，当 Client 发送的数据报文包丢失时，由于在发送时 Client 为每一个报文包设置了一个计时器，超时未接受到对应的 Ack 报文时，则会重传并重新计时。当重传次数超过最大次数后，则会认为和 Server 断开连接，退出程序。

当 Server 发送的 ACK 报文包丢失时，由于 Client 的超时重传，Server 会重新接收到这个重复的报文包，此时 Server 直接丢弃这个报文包并重新回复这个 ACK 报文。



程序实现过程

开始时，Client 和 Server 均设置了端口选择。通信过程如下图所示。



```
// Server
int main() {
    int RouterPORT, ServerPORT;
    cout << "Please Input Router Port: ";
    cin >> RouterPORT;
    cout << "Please Input Server Port: ";
    cin >> ServerPORT;

    ...
}

// Client
int main() {
    int RouterPORT, ClientPORT;
    cout << "Please Input Router Port: ";
    cin >> RouterPORT;
    cout << "Please Input Client Port: ";
    cin >> ClientPORT;

    ...
}
```

接下来是 Socket 的初始化，我们这里使用 UDP 协议。过程基本同 lab1 中 Socket 的初始化。先初始化 Winsock，然后创建 UDP 套接字，并将 UDP 套接字设置成非阻塞模式，接下来配置套接字地址，然后通过我们上面实现的连接协议进行连接。

```
// 以 Client 为例展示初始化过程
WSADATA wsaDataStruct;
int result = WSStartup(MAKEWORD(2, 2), &wsaDataStruct);
if (result != 0) {
    Print("Failed to initialize winsock");
    return -1;
}
if (wsaDataStruct.wVersion != MAKEWORD(2, 2)) {
    Print("Unsupported winsock version");
    WSACleanup();
    return -1;
}
Print("Initialized winsock successfully");

SOCKET clientSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (clientSocket == INVALID_SOCKET) {
    Print("Failed to create socket");
    return -1;
}
```

```

unsigned long mode = 1;
if (ioctlsocket(clientSocket, FIONBIO, &mode) != NO_ERROR) {
    Print("Failed to set socket to non-blocking mode");
    closesocket(clientSocket);
    return -1;
}
Print("Created socket successfully");

SOCKADDR_IN serverAddr;
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");
serverAddr.sin_port = htons(RouterPORT);

SOCKADDR_IN clientAddr;
clientAddr.sin_family = AF_INET;
clientAddr.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");
clientAddr.sin_port = htons(ClientPORT);

bind(clientSocket, (LPSOCKADDR)&clientAddr, sizeof(clientAddr));

bool connected = Three_Shakehands(clientSocket, serverAddr);
if (!connected) {
    Print("Failed to establish client connection");
    return -1;
}

```

断开连接时，运行我们上述实现的断开连接协议，然后判断是否断开成功，成功后清空 socket。

```

bool brokeed = Four_wavehands(clientSocket, serverAddr);
if (!brokeed) {
    Print("Failed to disconnect client");
    return -1;
}
closesocket(clientSocket);

```

Server

处理了文件名的接收、文件内容的接收、校验以及最终的文件保存过程。

文件名的接收与处理

循环使用 `recvfrom` 函数从 `serverSocket` 接收消息，存储在 `nameMessage` 中。如果接收到的数据长度 `recvByte > 0`，则说明有消息到达。同时调用 `nameMessage.check()` 校验消息完整性。通过后提取文件名和大小并打印文件信息。

```

while (1) {
    int recvByte = recvfrom(serverSocket, (char*)&nameMessage,
        sizeof(nameMessage), 0, (sockaddr*)&clientAddr, &AddrLen);
    if (recvByte > 0) {
        if (nameMessage.check() && (nameMessage.Seq == init_seq + 1) &&
            (nameMessage.flag & FileName)) {
            fileSize = nameMessage.size;
            for (int i = 0; nameMessage.data[i]; i++)
                fileName[i] = nameMessage.data[i];

```

```

        cout << Get_Time() << "    " << "[Receive]Filename: " << fileName <<
        ", Filesize: " << fileSize << endl;

        ...

    }

    ...

}

...

}

```

回复 ACK 消息

构造一个回复消息 replyMessage，用来确认文件名消息的接收：

- 设置源端口、目标端口、消息标志位（包含 ACK）、序列号和确认号。
- 调用 setCheck() 设置校验和，确保消息完整性。
- 使用 sendto 发送回复消息到客户端地址 clientAddr。

```

Message replyMessage;
replyMessage.SrcPort = ServerPORT;
replyMessage.DestPort = RouterPORT;
replyMessage.flag += ACK;
replyMessage.Seq = init_seq++;
replyMessage.Ack = nameMessage.Seq;
replyMessage.setCheck();
sendto(serverSocket, (char*)&replyMessage, sizeof(replyMessage), 0,
(sockaddr*)&clientAddr, sizeof(SOCKADDR_IN));

```

文件内容的接收

接收完整加载的数据包

调用 `rec_Message` 函数循环接收一个完整的 Message，将每个数据块的 data 内容逐字节拷贝到缓冲区 filemsg 的对应位置。

```

for (int i = 0; i < batchNum; i++) {
    Message dataMsg;
    if (rec_Message(dataMsg, serverSocket, clientAddr)) {
        cout << Get_Time() << "    " << i + 1 << " full load data message received
successfully." << endl;
    } else {
        cout << Get_Time() << "    " << i + 1 << " full load data message received
failedly." << endl;
        return -1;
    }
    for (int j = 0; j < Max_DATA_SIZE; j++) {
        filemsg[i * Max_DATA_SIZE + j] = dataMsg.data[j];
    }
}

```

接收未完整加载的数据包

如果有剩余数据（leftNum > 0），调用 `rec_Message` 接收一个不满的数据包。将剩余数据拷贝到缓冲区的相应位置。

```

if (leftNum > 0) {
    Message dataMsg;
    if (rec_Message(dataMsg, serverSocket, clientAddr))
        Print("Not full load data message received successfully.");
    else {
        Print("Not full load data message received failedly.");
        return -1;
    }
    for (int j = 0; j < leftNum; j++) {
        filemsg[batchNum * Max_DATA_SIZE + j] = dataMsg.data[j];
    }
}
}

```

文件写入与释放资源

- 打开以接收到的文件名为名称的文件，模式为二进制写入 ("wb")。
- 将 filemsg 缓冲区中的数据写入文件，确保文件大小与接收的大小一致。
- 关闭文件句柄，释放资源。
- 打印文件写入成功信息，并释放动态分配的 filemsg 内存。

```

FILE* outputfile;
outputfile = fopen(fileName, "wb");
if (filemsg != 0) {
    fwrite(filemsg, fileSize, 1, outputfile);
    fclose(outputfile);
}
Print("File has written successfully.");
delete[] filemsg;

```

Client

文件读取到逐批数据发送、校验、记录传输时间和吞吐量。

文件读取

提示用户输入要发送的文件名并存储到 filename。打开文件，模式为二进制 (ifstream::binary); 如果文件无法打开，打印错误信息并退出。动态分配缓冲区 filemsg 用于存储文件内容。使用循环逐字节读取文件数据，存入缓冲区 filemsg，并统计文件大小 fileSize。

```

// 输入文件名
string filename;
cout << "Enter the filename to send: ";
cin >> filename;

// 打开并读取文件内容
ifstream fin(filename.c_str(), ifstream::binary);
if (!fin) {
    Print("Unable to open file.");
    return -1;
}

// 读取文件数据到缓冲区
BYTE* filemsg = new BYTE[MaxFileSize];

```

```

unsigned int fileSize = 0;
BYTE byte = fin.get();
while (fin) {
    filemsg[fileSize++] = byte;
    byte = fin.get();
}
fin.close();

```

发送文件名和文件大小

构造文件名消息，调用 `send_Message`，将文件名消息发送到服务器。

```

// 构造文件名消息
Message nameMessage;
nameMessage.SrcPort = ClientPORT;
nameMessage.DestPort = RouterPORT;
nameMessage.size = fileSize;
nameMessage.flag += FileName;
nameMessage.Seq = ++init_seq;
for (int i = 0; i < realname.size(); i++)
    nameMessage.data[i] = realname[i];
nameMessage.data[realname.size()] = '\0';
nameMessage.setCheck();

// 发送到服务器
if (!send_Message(nameMessage, clientSocket, serverAddr)) {
    Print("Failed to send filename and file size");
    return -1;
}
Print("Filename and file size sent successfully");

```

发送文件内容

计算发送批次，先后发送完整数据包和非完整数据包。

```

// 发送完整数据包
for (int i = 0; i < batchNum; i++) {
    Message dataMsg;
    dataMsg.SrcPort = ClientPORT;
    dataMsg.DestPort = RouterPORT;
    dataMsg.Seq = ++init_seq;
    for (int j = 0; j < Max_DATA_SIZE; j++) {
        dataMsg.data[j] = filemsg[i * Max_DATA_SIZE + j];
    }
    dataMsg.setCheck();
    if (!send_Message(dataMsg, clientSocket, serverAddr)) {
        Print("Failed to send full data packet");
        return -1;
    }
    cout << Get_Time() << " " << "Full data packet " << i + 1 << " sent successfully" << endl << endl;
}

// 发送不完整数据包
if (leftNum > 0) {

```

```

Message dataMsg;
dataMsg.SrcPort = ClientPORT;
dataMsg.DestPort = RouterPORT;
dataMsg.Seq = ++init_seq;
for (int j = 0; j < leftNum; j++) {
    dataMsg.data[j] = filemsg[batchNum * Max_DATA_SIZE + j];
}
dataMsg.setCheck();
if (!send_Message(dataMsg, clientSocket, serverAddr)) {
    Print("Failed to send partial data packet");
    return -1;
}
Print("Partial data packet sent successfully");
}

```

计算传输时间和吞吐量

使用 `clock()` 函数记录程序运行时间，单位为毫秒。用总文件大小 `fileSize` 除以传输时间，计算每毫秒的传输字节数。

```

int endtime = clock();
cout << "Total transfer time: " << (endtime - starttime) << "ms" << endl;
cout << "Average throughput: " << ((float)fileSize) / (endtime - starttime) << "
bytes/ms" << endl << endl;

```

结果验证

本次实验的测试文件有 `1.jpg`、`2.jpg`、`3.jpg` 和 `helloworld.txt` 四个文件，接下来我将对这四个文件进行传输测试。

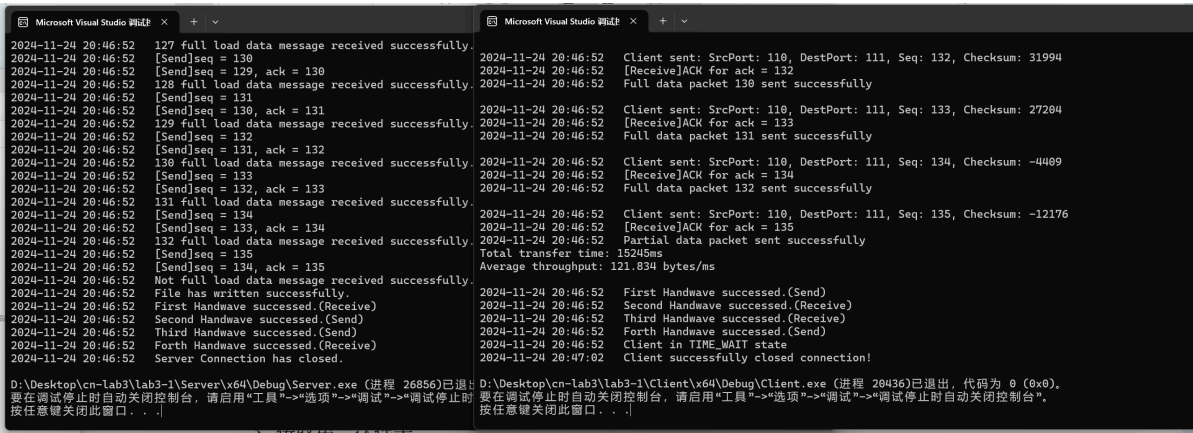
```

IP = "127.0.0.1";
ClientPORT = 110;
RouterPORT = 111;
ServerPORT = 112;

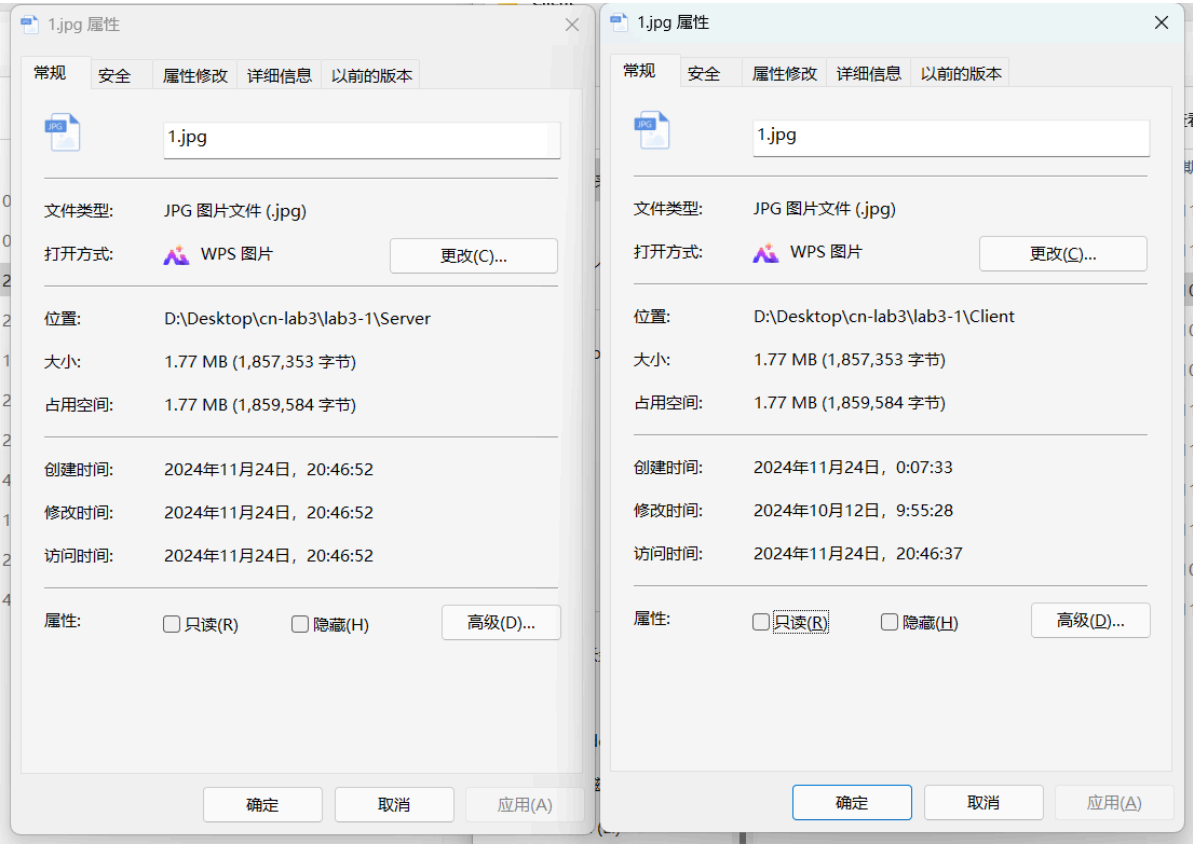
```

文件对比

1.jpg :



传输前后文件对比



2.jpg :

```
Microsoft Visual Studio 调试
2024-11-24 20:41:50 416 full load data message received successfully.
2024-11-24 20:41:50 [Send]seq = 419
2024-11-24 20:41:50 [Send]seq = 418, ack = 419
2024-11-24 20:41:50 417 full load data message received successfully.
2024-11-24 20:41:50 [Send]seq = 420
2024-11-24 20:41:50 [Send]seq = 419, ack = 420
2024-11-24 20:41:50 418 full load data message received successfully.
2024-11-24 20:41:50 [Send]seq = 421
2024-11-24 20:41:50 [Send]seq = 420, ack = 421
2024-11-24 20:41:50 419 full load data message received successfully.
2024-11-24 20:41:50 [Send]seq = 422
2024-11-24 20:41:50 [Send]seq = 421, ack = 422
2024-11-24 20:41:50 420 full load data message received successfully.
2024-11-24 20:41:51 [Send]seq = 423
2024-11-24 20:41:51 [Send]seq = 422, ack = 423
2024-11-24 20:41:51 421 full load data message received successfully.
2024-11-24 20:41:51 [Send]seq = 424
2024-11-24 20:41:51 [Send]seq = 423, ack = 424
2024-11-24 20:41:51 Not full load data message received successfully.
2024-11-24 20:41:51 File has written successfully.
2024-11-24 20:41:51 First Handwave succeeded.(Receive)
2024-11-24 20:41:51 Second Handwave succeeded.(Send)
2024-11-24 20:41:51 Third Handwave succeeded.(Send)
2024-11-24 20:41:51 Forth Handwave succeeded.(Receive)
2024-11-24 20:41:51 Server Connection has closed.

D:\Desktop\cn-lab3\lab3-1\Server\vs64\Debug\Server.exe (进程 7080)已退出
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时
按任意键关闭此窗口。...

Microsoft Visual Studio 调试
2024-11-24 20:41:50 Client sent: SrcPort: 110, DestPort: 111, Seq: 421, Checksum: 32681
2024-11-24 20:41:50 [Receive]ACK for ack = 421
2024-11-24 20:41:50 Full data packet 419 sent successfully
2024-11-24 20:41:50 Client sent: SrcPort: 110, DestPort: 111, Seq: 422, Checksum: -21314
2024-11-24 20:41:51 [Receive]ACK for ack = 422
2024-11-24 20:41:51 Full data packet 420 sent successfully
2024-11-24 20:41:51 Client sent: SrcPort: 110, DestPort: 111, Seq: 423, Checksum: -14375
2024-11-24 20:41:51 [Receive]ACK for ack = 423
2024-11-24 20:41:51 Full data packet 421 sent successfully
2024-11-24 20:41:51 Client sent: SrcPort: 110, DestPort: 111, Seq: 424, Checksum: 4359
2024-11-24 20:41:51 [Receive]ACK for ack = 424
2024-11-24 20:41:51 Partial data packet sent successfully
2024-11-24 20:41:51 Total transfer time: 3550ms
2024-11-24 20:41:51 Average throughput: 166.15 bytes/ms
2024-11-24 20:41:51 First Handwave succeeded.(Send)
2024-11-24 20:41:51 Second Handwave succeeded.(Receive)
2024-11-24 20:41:51 Third Handwave succeeded.(Receive)
2024-11-24 20:41:51 Forth Handwave succeeded.(Send)
2024-11-24 20:41:51 Client in TIME_WAIT state
2024-11-24 20:42:01 Client successfully closed connection!

D:\Desktop\cn-lab3\lab3-1\Client\vs64\Debug\Client.exe (进程 44108)已退出, 代码为 0 (0x0).
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。...]
```

传输前后文件对比

2.jpg 属性

常规 安全 属性修改 详细信息 以前的版本

2.jpg

文件类型: JPG 图片文件 (.jpg)

打开方式: WPS 图片 更改(C)...

位置: D:\Desktop\cn-lab3\lab3-1\Server

大小: 5.62 MB (5,898,505 字节)

占用空间: 5.62 MB (5,902,336 字节)

创建时间: 2024年11月24日, 20:41:51

修改时间: 2024年11月24日, 20:41:51

访问时间: 2024年11月24日, 20:41:51

属性: ☐ 只读(R) ☐ 隐藏(H) 高级(D)...

确定 取消 应用(A)

2.jpg 属性

常规 安全 属性修改 详细信息 以前的版本

2.jpg

文件类型: JPG 图片文件 (.jpg)

打开方式: WPS 图片 更改(C)...

位置: D:\Desktop\cn-lab3\lab3-1\Client

大小: 5.62 MB (5,898,505 字节)

占用空间: 5.62 MB (5,902,336 字节)

创建时间: 2024年11月24日, 0:07:33

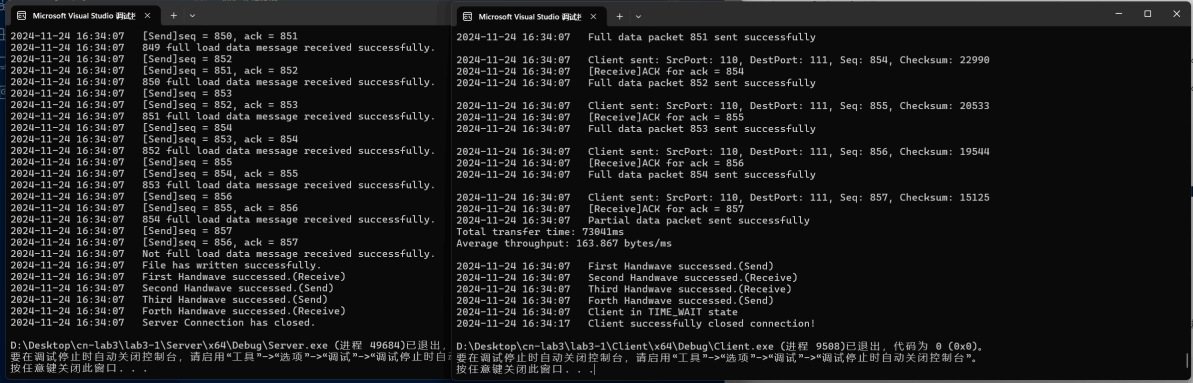
修改时间: 2024年10月12日, 9:55:26

访问时间: 2024年11月24日, 20:41:15

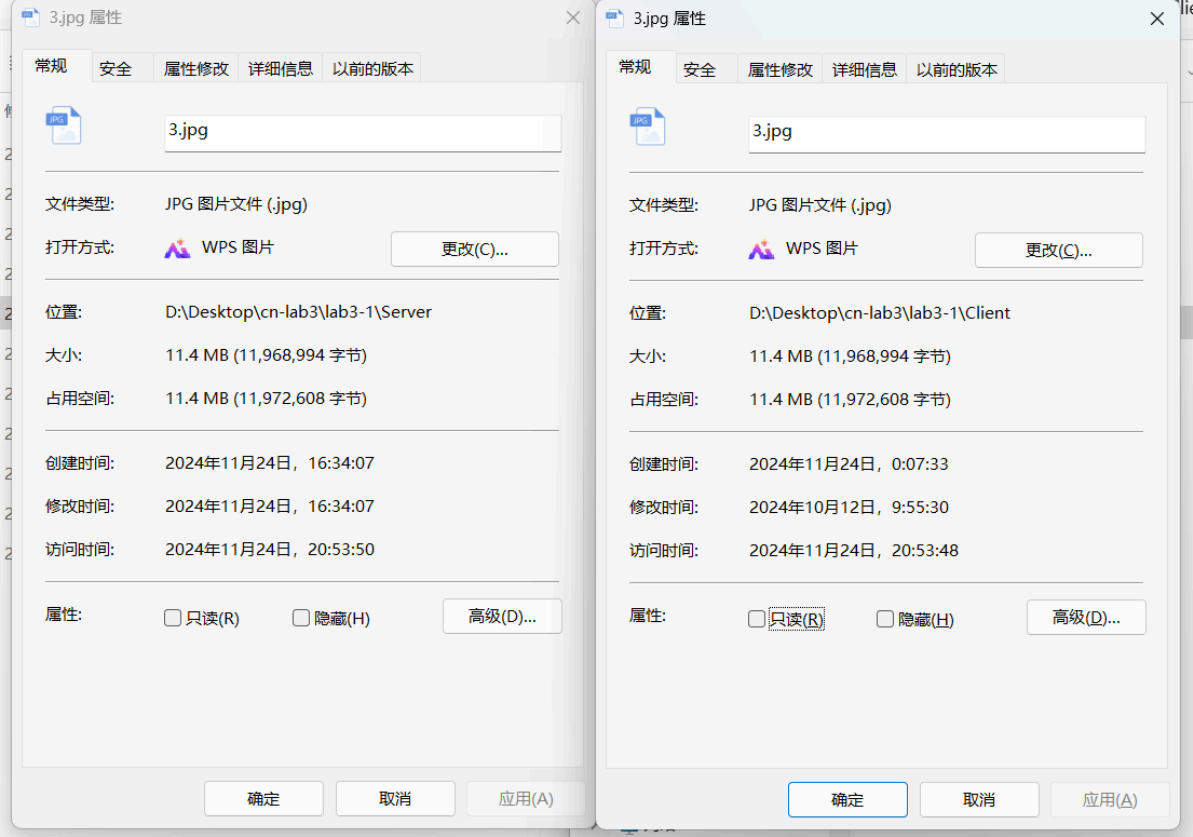
属性: ☐ 只读(R) ☐ 隐藏(H) 高级(D)...

确定 取消 应用(A)

3.jpg :



传输前后文件对比



helloworld.txt:

```
Microsoft Visual Studio 调试
2024-11-24 20:43:23 113 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 116
2024-11-24 20:43:23 [Send]seq = 115, ack = 116
2024-11-24 20:43:23 114 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 117
2024-11-24 20:43:23 [Send]seq = 116, ack = 117
2024-11-24 20:43:23 115 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 118
2024-11-24 20:43:23 [Send]seq = 117, ack = 118
2024-11-24 20:43:23 116 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 119
2024-11-24 20:43:23 [Send]seq = 118, ack = 119
2024-11-24 20:43:23 117 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 120
2024-11-24 20:43:23 [Send]seq = 119, ack = 120
2024-11-24 20:43:23 118 full load data message received successfully.
2024-11-24 20:43:23 [Send]seq = 121
2024-11-24 20:43:23 [Send]seq = 120, ack = 121
2024-11-24 20:43:23 Not full load data message received successfully.
2024-11-24 20:43:23 File has written successfully.
2024-11-24 20:43:23 First Handwave succeeded.(Receive)
2024-11-24 20:43:23 Second Handwave succeeded.(Send)
2024-11-24 20:43:23 Third Handwave succeeded.(Send)
2024-11-24 20:43:23 Forth Handwave succeeded.(Receive)
2024-11-24 20:43:23 Server Connection has closed.

D:\Desktop\cn-lab3\lab3-1\Server\x64\Debug\Server.exe (进程 54168)已退出
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时
任意键关闭此窗口。 . . |

Microsoft Visual Studio 调试
2024-11-24 20:43:23 Client sent: SrcPort: 110, DestPort: 111, Seq: 118, Checksum: -11700
2024-11-24 20:43:23 [Receive]ACK for ack = 118
2024-11-24 20:43:23 Full data packet 116 sent successfully
2024-11-24 20:43:23 Client sent: SrcPort: 110, DestPort: 111, Seq: 119, Checksum: 20455
2024-11-24 20:43:23 [Receive]ACK for ack = 119
2024-11-24 20:43:23 Full data packet 117 sent successfully
2024-11-24 20:43:23 Client sent: SrcPort: 110, DestPort: 111, Seq: 120, Checksum: -11702
2024-11-24 20:43:23 [Receive]ACK for ack = 120
2024-11-24 20:43:23 Full data packet 118 sent successfully
2024-11-24 20:43:23 Client sent: SrcPort: 110, DestPort: 111, Seq: 121, Checksum: 15621
2024-11-24 20:43:23 [Receive]ACK for ack = 121
2024-11-24 20:43:23 Partial data packet sent successfully
2024-11-24 20:43:23 Total transfer time: 9549ms
2024-11-24 20:43:23 Average throughput: 173.438 bytes/ms
2024-11-24 20:43:23 First Handwave succeeded.(Send)
2024-11-24 20:43:23 Second Handwave succeeded.(Receive)
2024-11-24 20:43:23 Third Handwave succeeded.(Receive)
2024-11-24 20:43:23 Forth Handwave succeeded.(Send)
2024-11-24 20:43:23 Client in TIME_WAIT state
2024-11-24 20:43:23 Client successfully closed connection!

D:\Desktop\cn-lab3\lab3-1\Client\x64\Debug\Client.exe (进程 39168)已退出，代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
任意键关闭此窗口。 . . |
```

传输前后文件对比

helloworld.txt 属性

常规

安全

详细信息

以前的版本

 helloworld.txt

文件类型: 文本文档 (.txt)

打开方式:  记事本

更改(C)...

位置: D:\Desktop\cn-lab3\lab3-1\Server

大小: 1.57 MB (1,655,808 字节)

占用空间: 1.58 MB (1,658,880 字节)

创建时间: 2024年11月24日, 20:43:23

修改时间: 2024年11月24日, 20:43:23

访问时间: 2024年11月24日, 20:43:23

属性: ☐ 只读(R) ☐ 隐藏(H)

高级(D)...

确定

取消

应用(A)

helloworld.txt 属性

常规

安全

详细信息

以前的版本

 helloworld.txt

文件类型: 文本文档 (.txt)

打开方式:  记事本

更改(C)...

位置: D:\Desktop\cn-lab3\lab3-1\Client

大小: 1.57 MB (1,655,808 字节)

占用空间: 1.58 MB (1,658,880 字节)

创建时间: 2024年11月24日, 0:07:33

修改时间: 2024年10月12日, 9:55:26

访问时间: 2024年11月24日, 20:43:14

属性: ☐ 只读(R) ☐ 隐藏(H)

高级(D)...

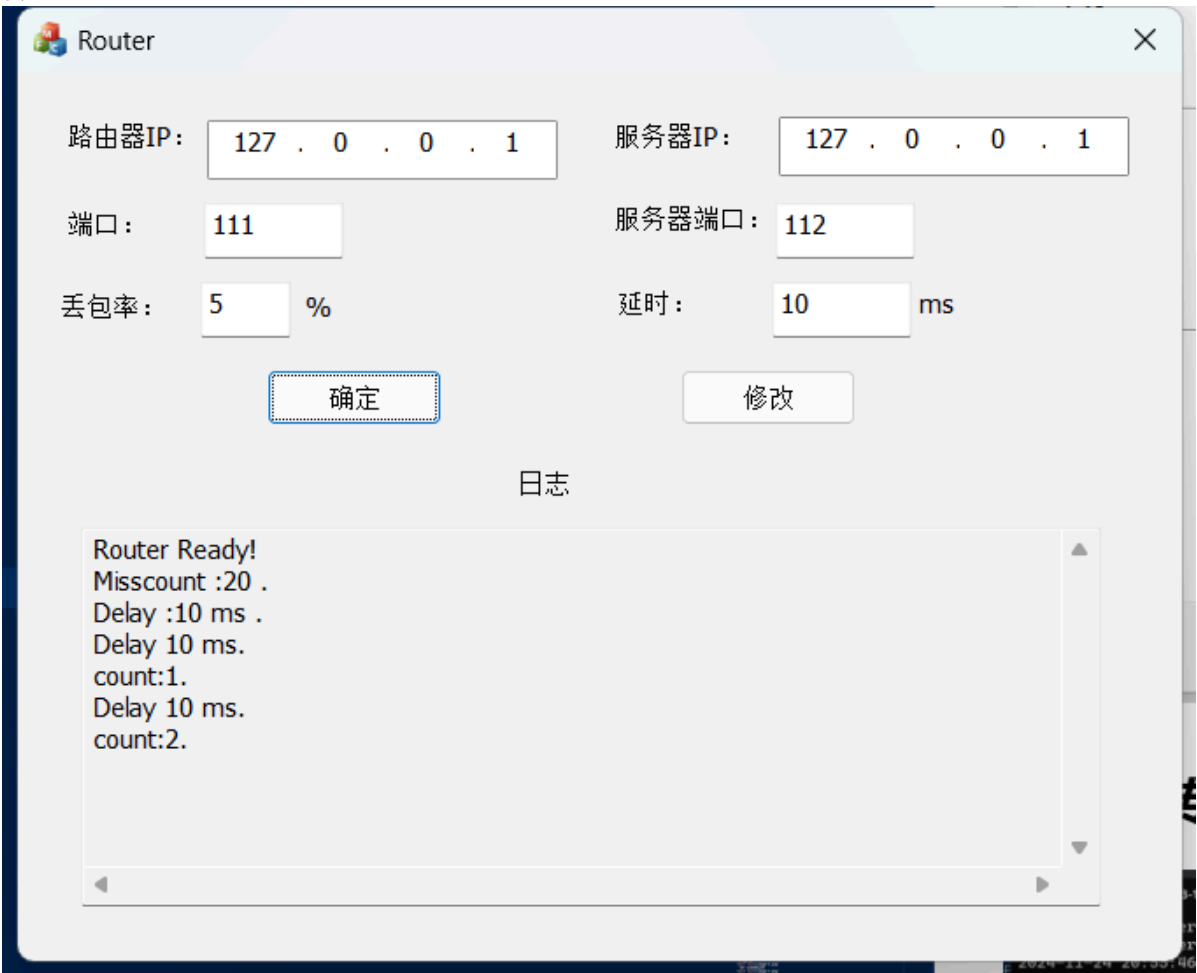
确定

取消

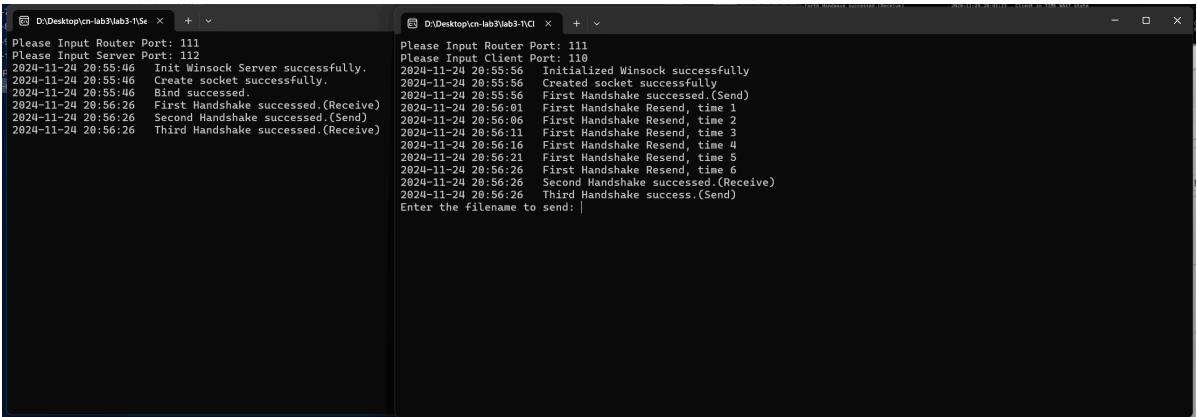
应用(A)

超时重传

未开启 Router 前，Client 的第一次握手请求无法发送到 Server 上，接收不到第二次握手，会超时重传。

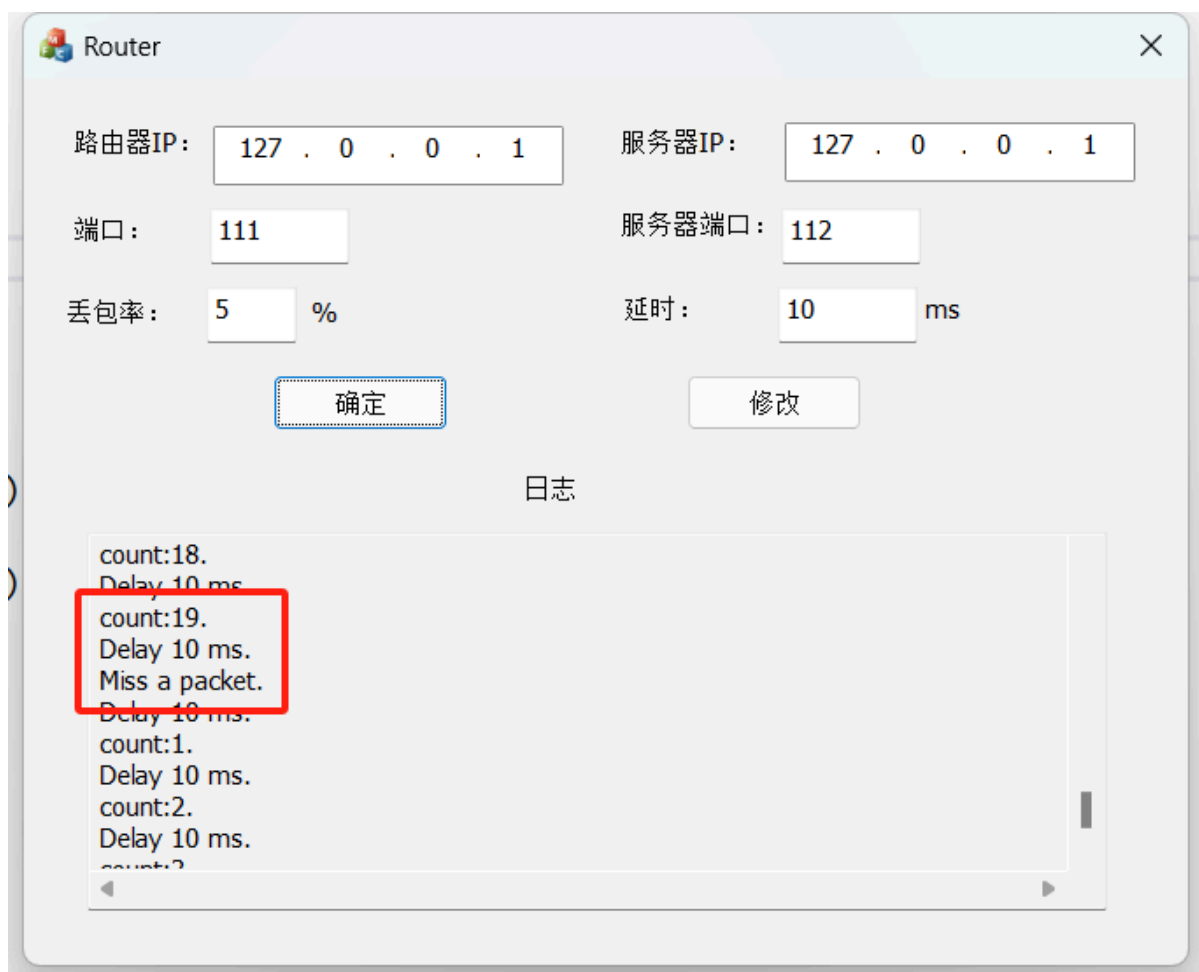


当打开 Router 并配置好后，Client 和 Server 之间成功连接。



这里我们以 1.jpg 为例，演示文件传输过程中的超时重传。

我们在 Router 的输出日志中成功捕获到丢包事件。



同时序列号 Seq = 114 的数据包出现丢失，根据终端输出，可知成功重传。

